NGINX

How It's Designed for Performance & Scale

Mohammad Mirsafaei
Elaheh Toulabi Nejad

Isfahan University Of Technology

Winter 2019

How normal Web servers handle connections

3 different behavior of workers:

1. Single listen socket, single worker processes
2. Single listen socket, multiple worker processes
3. Multiple worker processes, each with separate listen socket (socket sharding)

Blocking I/O

Most web application platforms use blocking (waiting) I/O

Each worker can only process one active connection at a time

Problems

Context switch overhead & waiting for client

Update

Reactor Pattern

Why NGINX ?

42.2% websites all over the world use NGINX web server

Lorem ipsum dolor sit amet, consectetur adipiscing elit.

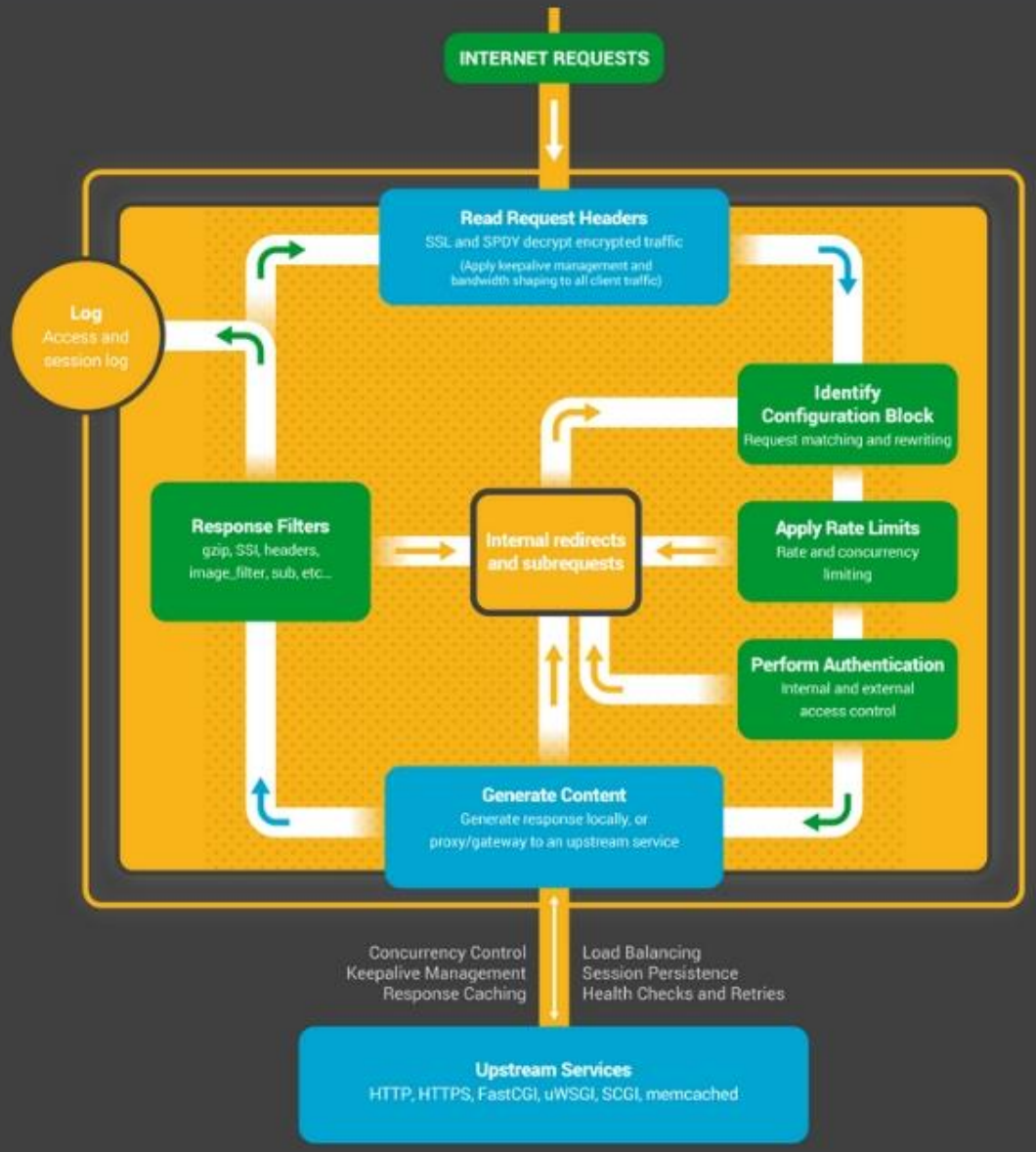
W3Techs.com, 22 December 2019

How NGINX handles connections ?

Master & Child Process

A closer look...

State machine



NGINX

A True Grandmaster !

NGINX uses a Non-Blocking "Event-Driven" architecture

Listen Sockets & Connection Sockets

Wait for an event (epoll or kqueue)

Event on Listen Socket:

- accept new
- set to be non-blocking
- add to the socket list

Event on Connection Socket:

- data in read buffer? read
- space in write buffer? write
- error or timeout? close & remove from socket list

An NGINX worker can process hundreds of thousands of active connections at the same time

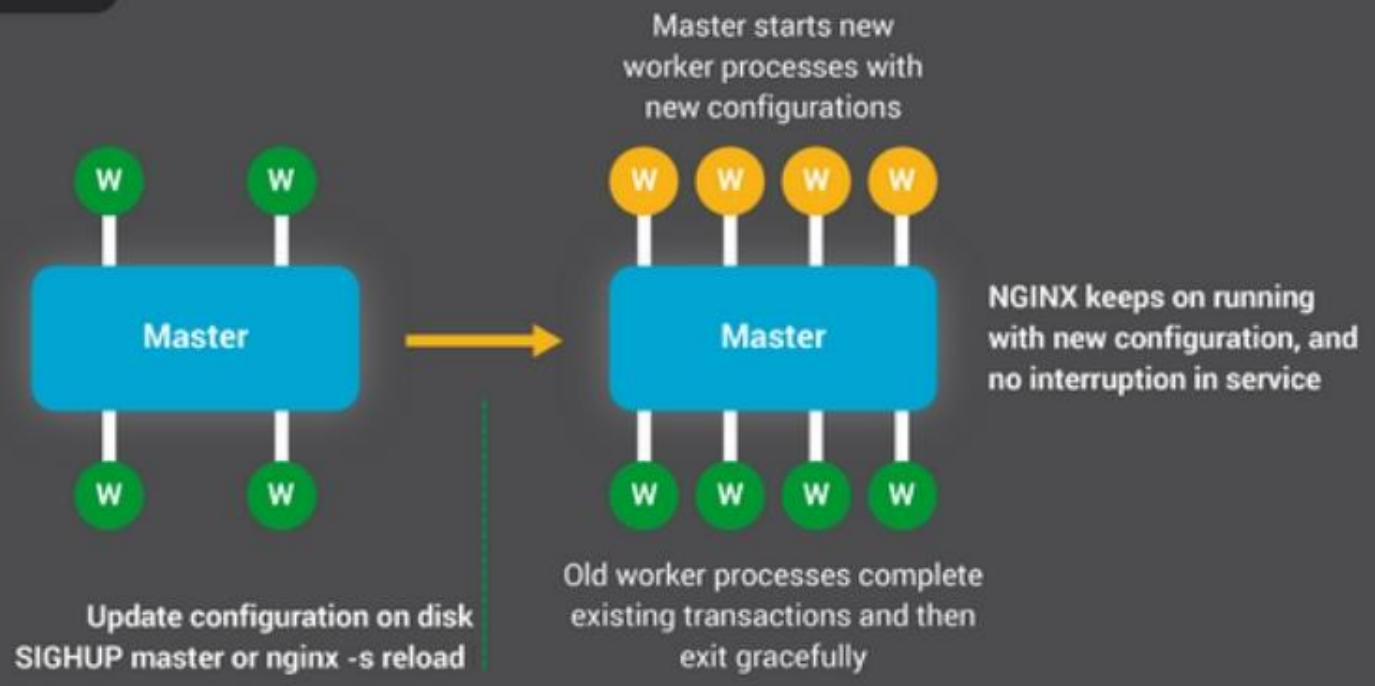
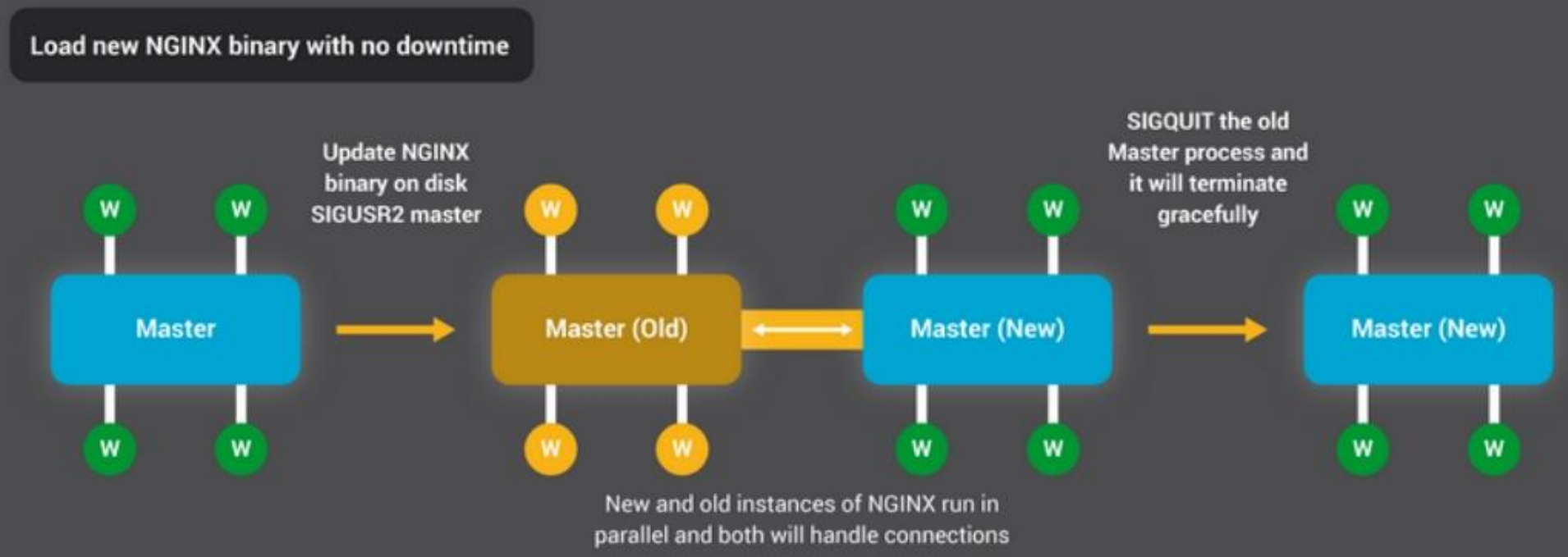
Faster than a Blocking, Multi process Architecture Because of:

1. less context switch overhead
2. handling thousands of concurrent connections with few workers

Signals in NGINX Update

Updating Configuration

Load new configuration with no downtime



Updating NGINX Binary File